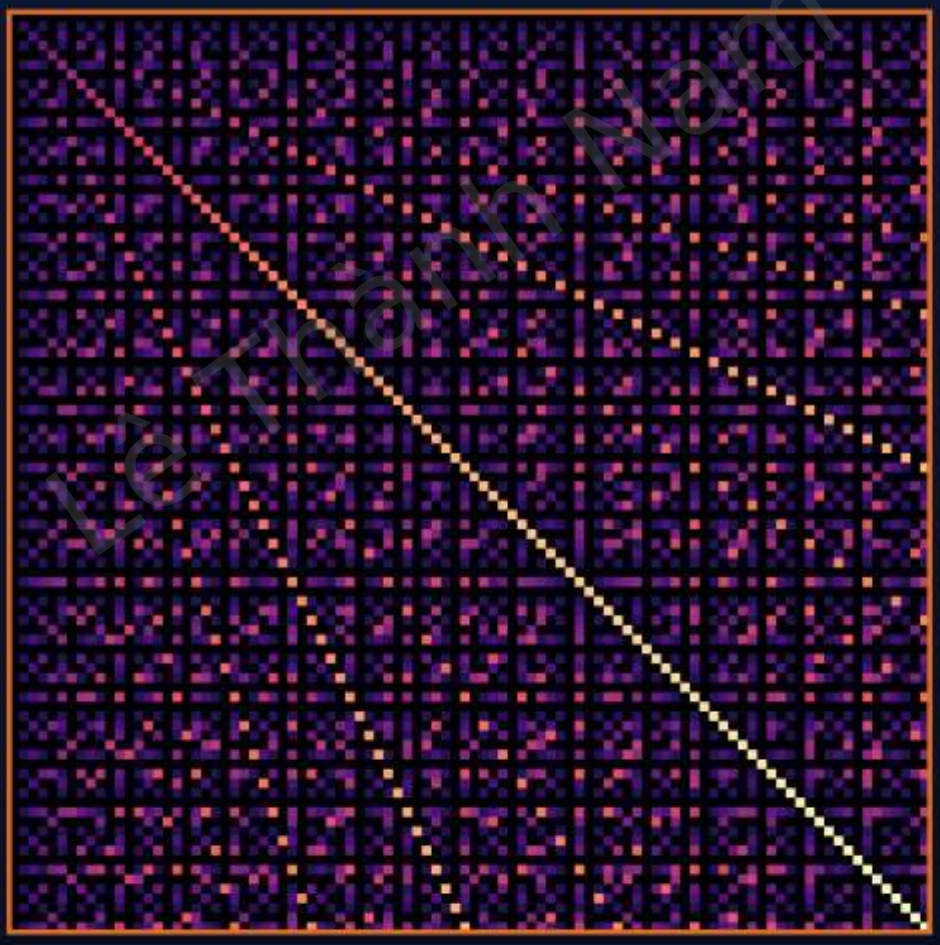
BỘ SÁCH BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

TẬP 1

CHIA HẾT · ƯỚC CHUNG LỚN NHẤT BỘI CHUNG NHỎ NHẤT



Ảnh nền: bảng gcd(i, j) với $1 \le i, j \le 96$, ô càng sáng ƯCLN càng lớn

Từ cách ngây thơ đến lời giải tối ưu · Chứng minh chi tiết · Mã nguồn C++ đã kiểm thử

Tổng hợp và biên soạn

LÊ THÀNH NAM

BỘ SÁCH BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

TUYỂN TẬP BÀI TẬP TOÁN TRÊN CODEFORCES

TẬP 1

CHIA HẾT, ƯỚC CHUNG LỚN NHẤT, BỘI CHUNG NHỎ NHẤT

Lê Thành Nam

Tổng hợp và biên soạn

LÊ THÀNH NAM

19 bài chọn lọc từ Codeforces, rating 800 đến 2100

Năm 2026

MỤC LỤC

Giới thiệu 5
Cuốn sách này dành cho ai 5
Cách giải 5
Subtask và các tầng lời giải 6
Sử dụng sách 6
Quy ước về chương trình 6
Ràng buộc và độ phức tạp 7
Bố cục sách 7
Chương 1. Chặn trên và xây dựng 8
Kiến thức cần nhớ 8
Các bài trong chương 9
Bài 1. ƯCLN lớn nhất 10
Bài 2. Tổng ƯCLN và BCNN 17
Bài 3. Bài toán ƯCLN 22
Bài 4. Các ước cách xa nhau 30
Chương 2. Bất biến và ƯCLN của các hiệu 37
Kiến thức cần nhớ 37
Các bài trong chương 37
Bài 5. Phép trừ liên tiếp 38
Bài 6. Đổi chỗ trong hoán vị 46
Bài 7. Đưa về bằng nhau 55
Bài 8. ƯCLN theo hàng 62
Chương 3. Duyệt ước 68
Kiến thức cần nhớ 68
Các bài trong chương 68
Bài 9. Cộng ước nhỏ nhất 69
Bài 10. Ước chung của dãy 75
Bài 11. Mua xẻng 81

Lê Thành Nam

Bài 12. Tách BCNN 87
Bài 13. Cặp số biết ƯCLN và BCNN 95
Chương 4. Số mũ nguyên tố 103
Kiến thức cần nhớ 103
Các bài trong chương 103
Bài 14. ƯCLN của các BCNN 104
Bài 15. Chia đúng k lượt 111
Bài 16. Tăng ƯCLN 119
Chương 5. Hàm Euler và sàng 127
Kiến thức cần nhớ 127
Các bài trong chương 127
Bài 17. ƯCLN giữ nguyên 128
Bài 18. Hai ước 134
Bài 19. Số cặp 142
Phụ lục 150
Bảng tổng hợp 150
Bài tập luyện thêm 151

Lê Thành Nam

Giới thiệu

Cuốn sách này dành cho ai

Sách gồm 19 bài về **chia hết, ước chung lớn nhất và bội chung nhỏ nhất** chọn từ kho đề Codeforces, xếp thành năm chương theo mạch kiến thức: chặn trên và xây dựng; bất biến và ƯCLN của các hiệu; duyệt ước; số mũ nguyên tố; hàm Euler và sàng. Độ khó Codeforces trải từ 800 đến 2100: bài dễ nhất dành cho học sinh mới làm quen với số học trong lập trình, bài khó nhất ngang tầm đề thi học sinh giỏi cấp tỉnh và vòng loại quốc gia.

Mỗi bài gồm hai phần. **Đề bài** được viết lại bằng tiếng Việt, giữ đúng yêu cầu, ràng buộc và ví dụ của đề gốc, kèm **bảng subtask** do sách bổ sung. **Hướng dẫn giải** không đưa ngay lời giải tốt nhất mà đi qua nhiều **tầng**: lời giải ngây thơ, các tầng cải tiến, rồi mới đến lời giải tối ưu. Mỗi tầng đều có chương trình chạy được, có thời gian chạy đo thật và có hình minh họa.

> **Về cách viết đề.** Đề trong sách không phải bản dịch từng câu. Mỗi đề được viết lại theo cách hiểu tự nhiên của người Việt, bỏ những chi tiết kể chuyện không ảnh hưởng đến lời giải. Mọi điều kiện, giới hạn và ví dụ được giữ nguyên. Mỗi bài ghi mã bài và đường dẫn gốc để học sinh mở đề và nộp bài.

Cách giải

Hướng dẫn giải của mỗi bài chia thành các phần có dải màu riêng. Trong mỗi phần, lời giải đi từng bước nhỏ (Bước 1, Bước 2, ...), và hầu hết các bước đều có hình minh họa.

Phần	Nội dung
HIỂU ĐỀ	Vẽ ra các trường hợp để hiểu chính xác yêu cầu; nêu các nhận xét đầu tiên
NGÂY THƠ	Tầng 1: ý tưởng tự nhiên nhất, sơ đồ khối, chương trình vét cạn và độ phức tạp
BẾ TẮC	Thời gian chạy đo thật của tầng 1, chỉ ra nguyên nhân chậm, kết thúc bằng một câu hỏi then chốt
CẢI TIẾN	Các tầng trung gian: mỗi tầng sửa đúng một điểm yếu của tầng trước, có chương trình và subtask vượt qua được
LÓE SÁNG	Tính chất toán học quyết định dẫn tới lời giải tối ưu, có chứng minh và hình minh họa
TỐI ƯU	Chương trình tối ưu, độ phức tạp và bảng so sánh tất cả các tầng trên mọi subtask

Phần	Nội dung
LƯU Ý	Những chỗ dễ sai và phần mở rộng

Thời gian chạy trong sách được đo thật bằng trình biên dịch g++ 13 (tùy chọn -O2) trên máy chủ Linux, tính cả thời gian đọc dữ liệu. Trên máy chấm khác các con số có thể khác đôi chút, nhưng tỉ lệ giữa các tầng là như nhau.

Subtask và các tầng lời giải

Đề thi học sinh giỏi trong nước thường chia test thành nhiều **subtask** (nhóm test) với ràng buộc tăng dần. Vì vậy mỗi bài trong sách có một bảng subtask: subtask đầu dành cho lời giải ngây thơ, các subtask giữa dành cho các tầng cải tiến, subtask cuối thường là ràng buộc gốc của Codeforces. Với những bài mà đề gốc có ràng buộc nhỏ, sách **mở rộng** ràng buộc ở subtask cuối để lời giải tối ưu thật sự cần thiết.

Cuối mỗi bài có bảng so sánh: mỗi tầng chạy trên test lớn nhất của từng subtask, ghi thời gian thật và số điểm đạt được. Khi luyện thi, học sinh nên viết **ngay** tầng ngây thơ để lấy điểm subtask nhỏ và để có chương trình đối chiếu, rồi mới nâng cấp dần.

Sử dụng sách

Với mỗi bài, học sinh nên đọc đề, tự nghĩ ít nhất 20 đến 30 phút rồi mới đọc hướng dẫn. Khi đọc hướng dẫn, nên dừng sau mỗi khung **Câu hỏi then chốt** và tự trả lời trước khi đọc tiếp.

Mọi chương trình trong sách đã được chạy với ví dụ của đề và đối chiếu chéo với nhau trên 60 test ngẫu nhiên mỗi bài; các bài có nhiều đáp án được kiểm tra bằng chương trình chấm riêng. Chương trình tối ưu đã được đo trên test lớn nhất theo ràng buộc.

Quy ước về chương trình

- Viết bằng C++14, chỉ dùng các thư viện chuẩn cụ thể (iostream, cstdio, vector, algorithm, set), **không dùng** tệp bits/stdc++.h để biên dịch được trên phần mềm chấm Themis.
- Viết đầy đủ, không viết tắt: không macro, không typedef, mỗi câu lệnh một dòng, mọi khối lệnh có dấu ngoặc nhọn. Chương trình không chứa chú thích; mọi giải thích nằm trong phần hướng dẫn.
- Tên biến và tên hàm đơn giản, gần với công thức: **ucln**, **laNguyenTo**, **dem**, **kq**, **best**.

- Đọc ghi tệp **có điều kiện**: chương trình chỉ chuyển sang tệp CFxxxx.INP và CFxxxx.OUT khi tệp dữ liệu vào tồn tại. Nhờ vậy cùng một chương trình chạy được trên Themis và khi nộp lên Codeforces (đọc từ bàn phím).

Hàm ƯCLN dùng chung trong cả cuốn sách là thuật toán Euclid. Hàm chấp nhận cả số âm và luôn trả về giá trị không âm, quy ước $\text{ƯCLN}(a, 0) = |a|$:

```
long long ucln(long long a, long long b) {
    if (a < 0) {
        a = -a;
    }
    if (b < 0) {
        b = -b;
    }
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}
```

Ràng buộc và độ phức tạp

Ràng buộc quyết định độ phức tạp được phép. Bảng dưới đây là "la bàn" cho mọi bài trong sách, với giả định máy chấm thực hiện khoảng 10^8 phép tính đơn giản mỗi giây.

Ràng buộc n	Độ phức tạp tối đa	Kĩ thuật thường gặp
≤ 20	$O(2^n \cdot n)$	vét cạn tập con
≤ 5000	$O(n^2)$	duyệt cặp
$\leq 10^5$ đến 10^6	$O(n\sqrt{n}), O(n\log n)$	gộp ƯCLN, duyệt bội
$\leq 10^7$	$O(n\log\log n)$	sàng Eratosthenes, sàng đếm
$\leq 10^{12}$	$O(\sqrt{n})$	duyệt ước tới căn bậc hai
$\leq 10^{18}$	$O(\log n)$ hoặc $O(1)$	thuật toán Euclid, công thức

Bố cục sách

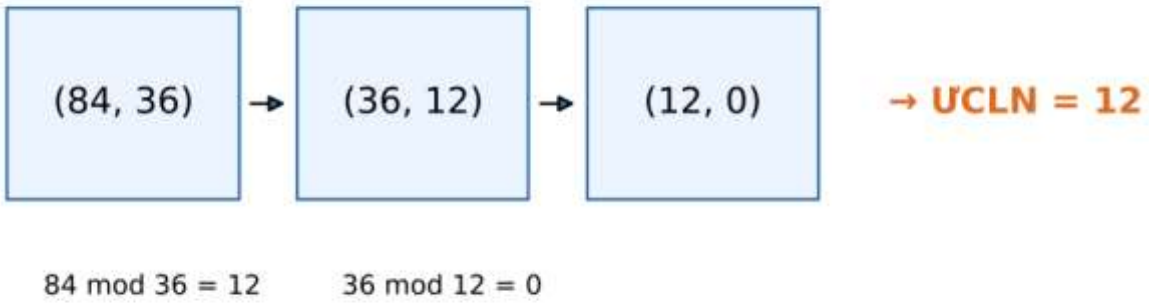
Chương	Chủ đề	Bài (mã Codeforces)
1	Chặn trên và xây dựng	1370A, 1325A, 1617B, 1474B
2	Bất biến và ƯCLN của các hiệu	1708A, 1828B, 1593D1, 1458A
3	Duyệt ước	1350A, 1203C, 1360D, 1285C, 992B
4	Số mũ nguyên tố	1349A, 1538D, 1047C
5	Hàm Euler và sàng	1295D, 1366D, 1499D

Chương 1. Chặn trên và xây dựng

Trọng tâm chương. Bốn bài trong chương đều có lời giải rất ngắn, nhưng để tìm ra cần một **nhận xét**. Khi đề hỏi giá trị lớn nhất, ta tìm **chặn trên** rồi chỉ ra một cách chọn đạt đúng chặn trên đó. Khi đề chỉ cần **một** nghiệm, ta thử các giá trị đặc biệt (1, 2, số nguyên tố, hai số lẻ liền nhau) trước khi nghĩ tới thuật toán phức tạp. Chương này cũng rèn thói quen viết ngay lời giải vét cạn để đối chiếu.

Kiến thức cần nhớ

- **Chia hết.** $b \mid a$ nếu tồn tại số nguyên k với $a = k \cdot b$. Khi đó b là ước của a, a là bội của b.
- **ƯCLN và BCNN.** $\text{ƯCLN}(a, b)$ là số nguyên dương lớn nhất chia hết cả a và b; $\text{BCNN}(a, b)$ là số nguyên dương nhỏ nhất chia hết cho cả hai. Với a, b nguyên dương: $\text{ƯCLN}(a, b) \cdot \text{BCNN}(a, b) = a \cdot b$. Khi tính BCNN, luôn chia trước rồi nhân sau: $\frac{a}{\text{ƯCLN}(a,b)} \cdot b$.
- **Thuật toán Euclid.** $\text{ƯCLN}(a, b) = \text{ƯCLN}(b, a \bmod b)$, dừng khi số thứ hai bằng 0. Số bước là $O(\log\min(a, b))$.
- **Tính chất hiệu.** Với mọi số nguyên k: $\text{ƯCLN}(a, b) = \text{ƯCLN}(a, b - k \cdot a)$. Mọi ước chung của a và b đều chia hết hiệu $b - a$.
- **Số 1 và số nguyên tố.** $\text{ƯCLN}(1, b) = 1$, $\text{BCNN}(1, b) = b$. Ước nhỏ nhất lớn hơn 1 của một số luôn là số nguyên tố.



Hình 0.1. Chạy tay thuật toán Euclid với cặp (84, 36)

Các bài trong chương

Bài	Tên bài	Mã Codeforces	Độ khó	Số tầng
1	ƯCLN lớn nhất (Maximum GCD)	1370A	800	3
2	Tổng ƯCLN và BCNN (EhAb AnD gCd)	1325A	800	2
3	Bài toán ƯCLN (GCD Problem)	1617B	900	3
4	Các ước cách xa nhau (Different Divisors)	1474B	1000	2

Bài 1. ƯCLN lớn nhất

Codeforces 1370A - Maximum GCD Độ khó: 800

Đề gốc: https://codeforces.com/problemset/problem/1370/A

Đề bài

Cho số nguyên n. Xét tất cả các cặp số nguyên **phân biệt** a, b thuộc đoạn $[1, n]$. Hãy tìm giá trị lớn nhất của ƯCLN(a, b).

Dữ liệu vào: Dòng đầu chứa số test t ($1 \le t \le 100$). Mỗi dòng tiếp theo chứa một số nguyên n ($2 \le n \le 10^6$).

Kết quả: Với mỗi test, in ra giá trị lớn nhất của ƯCLN(a, b) trên một dòng.

Ví dụ

Dữ liệu vào	Kết quả
2 3 5	1 2

Giải thích ví dụ. Với $n = 3$, mọi cặp trong $\{1,2,3\}$ đều có ƯCLN bằng 1. Với $n = 5$, cặp $(2,4)$ có ƯCLN bằng 2 và không cặp nào tốt hơn.

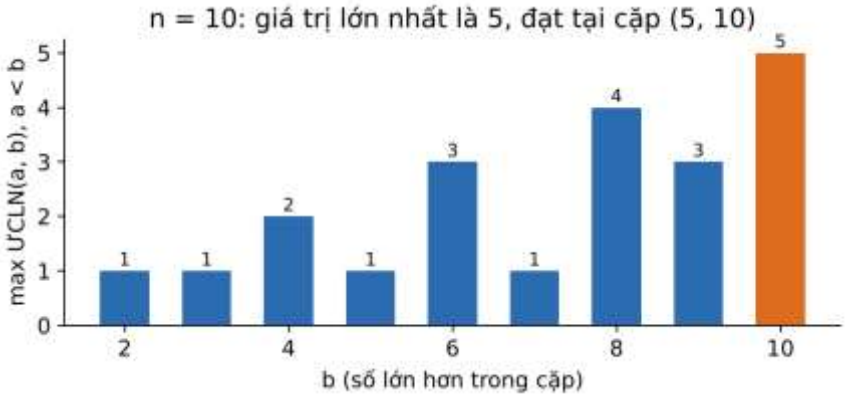
Subtask

Các subtask do sách bổ sung để luyện từng mức thuật toán; subtask chứa ràng buộc gốc được ghi chú, các subtask lớn hơn là mở rộng. Giới hạn thời gian 2 giây cho mọi subtask.

Subtask	Ràng buộc	Điểm
ST1	$t \le 10, n \le 1000$	30%
ST2	$t \le 100, n \le 10^6$ (ràng buộc gốc)	30%
ST3	$t \le 10^5, n \le 10^{18}$	40%

Hướng dẫn giải

HIỂU ĐỀ Hiểu đúng đề bài



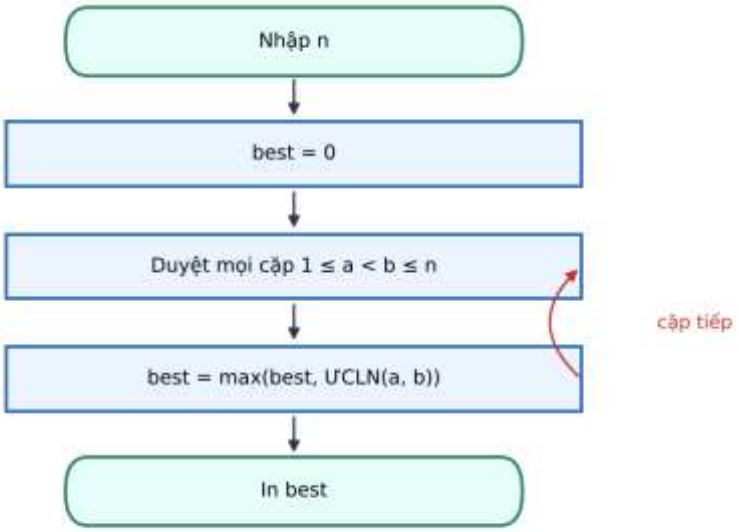
Hình 1.1. Với từng giá trị b, ƯCLN lớn nhất có thể đạt khi a < b (n = 10)

Nếu ƯCLN$(a, b) = d$ thì cả a và b đều là **bội** của d. Vậy câu hỏi "ƯCLN lớn nhất là bao nhiêu" có thể dịch thành: tìm số d lớn nhất sao cho đoạn $[1, n]$ chứa **ít nhất hai bội khác nhau** của d. Cách nhìn này chuyển bài toán từ "tính ƯCLN" sang "đếm bội", vốn dễ hơn nhiều.

NGÂY THƠ Ý tưởng ngây thơ: duyệt mọi cặp (a, b)

Bước 1. Ý tưởng

Duyệt mọi cặp (a, b) với $1 \le a < b \le n$, tính ƯCLN(a, b) bằng thuật toán Euclid và giữ lại giá trị lớn nhất.



Hình 1.2. Sơ đồ khối của tầng 1

Lê Thành Nam

Chạy tay với $n = 5$:

Cặp	(1,2)	(1,3)	(1,4)	(1,5)	(2,3)	(2,4)	(2,5)	(3,4)	(3,5)	(4,5)
ƯCLN	1	1	1	1	1	**2**	1	1	1	1

Bước 2. Cài đặt

Tầng 1: duyệt mọi cặp

```
#include <iostream>
#include <cstdio>
using namespace std;

long long ucln(long long a, long long b) {
    if (a < 0) {
        a = -a;
    }
    if (b < 0) {
        b = -b;
    }
    while (b != 0) {
        long long r = a % b;
        a = b;
        b = r;
    }
    return a;
}

int main() {
    FILE *f = fopen("CF1370A.INP", "r");
    if (f != NULL) {
        fclose(f);
        freopen("CF1370A.INP", "r", stdin);
        freopen("CF1370A.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int t;
    cin >> t;
    while (t > 0) {
        t--;
        long long n;
        cin >> n;
        long long best = 0;
        for (long long a = 1; a <= n; a++) {
            for (long long b = a + 1; b <= n; b++) {
                long long g = ucln(a, b);
                if (g > best) {
                    best = g;
                }
            }
        }
        cout << best << "\n";
    }
    return 0;
}
```

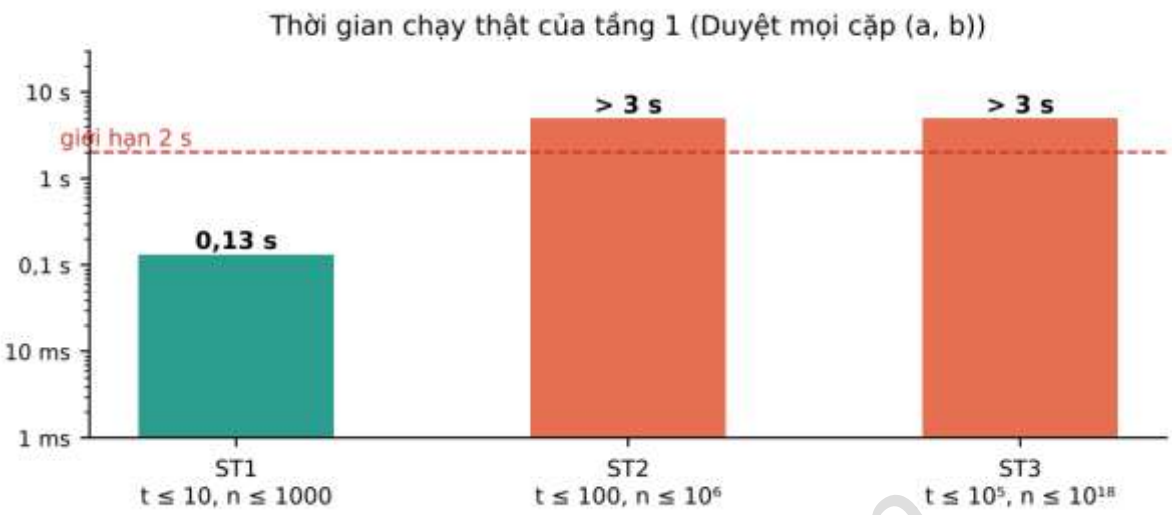
Lê Thành Nam

Bước 3. Đánh giá độ phức tạp

Có $\frac{n(n-1)}{2}$ cặp, mỗi cặp tốn $O(\log n)$ cho thuật toán Euclid. Tổng cộng $O(n^2 \log n)$ cho mỗi test.

BẾ TẮC Bế tắc: số cặp tăng theo bình phương

Bước 1. Đo thời gian thật



Hình 1.3. Thời gian chạy thật của tầng 1 trên test lớn nhất của từng subtask

Với $n = 1000$, tầng 1 chạy khoảng 0,13 giây cho 10 test. Với $n = 10^6$, số cặp là khoảng $5 \cdot 10^{11}$, nhân thêm 100 test thì không thể chạy xong.

Bước 2. Tìm nguyên nhân

Tầng 1 tính ƯCLN của mọi cặp, trong khi ta chỉ cần **giá trị lớn nhất**. Phần lớn các cặp có ƯCLN nhỏ và không bao giờ là đáp án.

> **Câu hỏi then chốt.** Thay vì duyệt cặp (a, b), nếu duyệt thẳng **giá trị** d của ƯCLN thì cần kiểm tra điều gì?

CẢI TIẾN 1 Cải tiến 1: duyệt d giảm dần, đếm bội

Bước 1. Ý tưởng

Theo phần Hiểu đề, d là ƯCLN của một cặp nào đó khi đoạn $[1, n]$ chứa ít nhất hai bội của d (chọn $a = d$, $b = 2d$ thì ƯCLN$(a, b) = d$). Số bội của d trong $[1, n]$ là $\left\lfloor \frac{n}{d} \right\rfloor$. Vậy duyệt d từ n giảm dần, dừng ở d đầu tiên có $\left\lfloor \frac{n}{d} \right\rfloor \geq 2$.

Lê Thành Nam

Bước 2. Cài đặt

Tầng 2: duyệt d giảm dần

```
#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    FILE *f = fopen("CF1370A.INP", "r");
    if (f != NULL) {
        fclose(f);
        freopen("CF1370A.INP", "r", stdin);
        freopen("CF1370A.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int t;
    cin >> t;
    while (t > 0) {
        t--;
        long long n;
        cin >> n;
        long long d = n;
        while (n / d < 2) {
            d--;
        }
        cout << d << "\n";
    }
    return 0;
}
```

Bước 3. Kết quả

Vòng lặp chạy khoảng $\frac{n}{2}$ lần, tức $O(n)$ mỗi test: qua ST1 và ST2. Nhưng nếu mở rộng tới $n = 10^{18}$ thì vòng lặp chạy $5 \cdot 10^{17}$ lần.

> **Câu hỏi then chốt.** Vòng lặp của tầng 2 luôn dừng ở cùng một vị trí. Có thể **tính thẳng** vị trí đó không?

LÓE SÁNG **Lóe sáng: chặn trên và cách xây dựng**

Bước 1. Tìm chặn trên

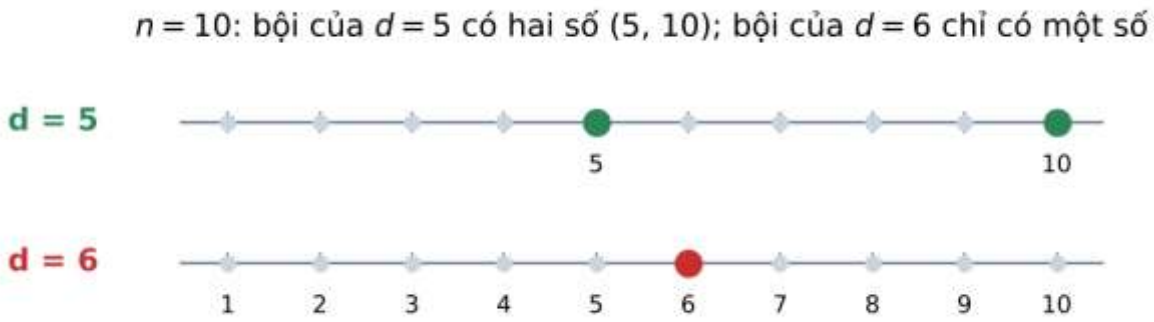
Giả sử $\text{ƯCLN}(a, b) = d$ với $a < b \le n$. Vì a và b là hai bội **khác nhau** của d, bội nhỏ nhất có thể là d, bội lớn hơn tiếp theo ít nhất là $2d$. Do đó $b \ge 2d$. Kết hợp với $b \le n$:

$$2d \le n \;\Rightarrow\; d \le \left\lfloor \frac{n}{2} \right\rfloor$$

Bước 2. Chứng tỏ chặn trên đạt được

Chọn $d = \left\lfloor \frac{n}{2} \right\rfloor$, $a = d$, $b = 2d$. Vì $2d \le n$ nên b thuộc đoạn $[1, n]$, và $a \ne b$. Ta có $\text{ƯCLN}(d, 2d) = d$.

Lê Thành Nam



Hình 1.4. Với n = 10: d = 5 có hai bội trong đoạn, còn d = 6 chỉ có một bội

Đáp án là $\left\lfloor \frac{n}{2} \right\rfloor$. Kiểm tra: $n = 5$ cho 2, khớp cặp (2,4); $n = 10$ cho 5, khớp cặp (5,10).

TỐI ƯU Lời giải tối ưu

Bước 1. Cài đặt

Tầng 3: công thức

```
#include <iostream>
#include <cstdio>
using namespace std;

int main() {
    FILE *f = fopen("CF1370A.INP", "r");
    if (f != NULL) {
        fclose(f);
        freopen("CF1370A.INP", "r", stdin);
        freopen("CF1370A.OUT", "w", stdout);
    }
    ios::sync_with_stdio(false);
    cin.tie(NULL);
    int t;
    cin >> t;
    while (t > 0) {
        t--;
        long long n;
        cin >> n;
        cout << n / 2 << "\n";
    }
    return 0;
}
```

Lê Thành Nam

Bước 2. Độ phức tạp

$O(1)$ cho mỗi test. Phép chia hai số nguyên n / 2 trong C++ tự lấy phần nguyên.

Bước 3. So sánh các tầng

Tầng	Ý tưởng	Độ phức tạp	ST1	ST2	ST3	Điểm
1	Duyệt mọi cặp (a, b)	O(n² log n)	0,13 s	quá giờ	quá giờ	30
2	Duyệt d giảm dần, đếm bội	O(n)	3,0 ms	0,13 s	quá giờ	60
3	Công thức n/2	O(1)	2,4 ms	3,0 ms	0,02 s	100

Thời gian đo thật (g++ -O2) trên test lớn nhất của từng subtask; giới hạn 2 giây. Ô đỏ: quá giờ, tràn bộ nhớ hoặc sai. Cột Điểm cộng phần trăm của các subtask mà tầng đó vượt qua.

LƯU Ý Những chỗ dễ sai

- Quên điều kiện $a \neq b$: nếu cho phép $a = b$ thì đáp án là n.
- Khi mở rộng n tới 10^{18} phải đọc bằng kiểu long long.
- Mở rộng: ƯCLN lớn nhất của k **số phân biệt** trong $[1, n]$ là $\left\lfloor \frac{n}{k} \right\rfloor$, vì cần k bội $d, 2d, \ldots, k \cdot d$.
- Mở rộng: nếu các số được chọn trong đoạn $[l, r]$ thì cần d lớn nhất thỏa $\left\lfloor \frac{r}{d} \right\rfloor - \left\lfloor \frac{l-1}{d} \right\rfloor \geq 2$.

Lê Thành Nam